

DAĞITIK SİSTEMLER

İşletim Sisteminin 4 Temel Görevi

- Donanım Arayüzü (Interface to peripheral hardware):** Bilgisayara bağlı olan fare, klavye, yazıcı veya ekran gibi cihazların (donanımların) yazılımla konuşmasını sağlar. Sen fareyi oynattığında, o hareketi işlemcinin anlayacağı dile işletim sistemi çevirir.
- Görev Zamanlama (Schedules tasks):** Bilgisayarda aynı anda birçok program çalışır. İşletim sistemi, işlemcinin (CPU) hangi programa ne kadar süre ayıracağına karar verir. Bir programın diğerini dondurmasını engeller.
- Depolama Alanı Yönetimi (Allocates storage):** Verilerin RAM (bellek) veya sabit diskte nereye yazılacağını ve ne kadar yer kaplayacağını yönetir. Hafızanın verimli kullanılmasını sağlar.
- Varsayılan Arayüz (Default interface):** Hiçbir uygulama (Chrome, oyun, hesap makinesi vb.) açık değilken bile karşında gördüğün masaüstü veya komut satırı ekranıdır. Bilgisayarı kullanman için sana her zaman hazır bir "çalışma alanı" sunar.

Dağıtık Sistem:

- Birden fazla bilgisayarın, ortak bir amaca yönelik, bir ağ üzerinden otonom çalışan, merkezi bir saat olmaksızın asenkron haberleşen, bileşenleri hata yapmaya meyilli, programlanabilir, birbirinden bağımsız çalışan yapılar.

Dağıtık sistem örnekleri: Web, internet, kablosuz ağ, DNS, Gnutella, BitTorrent, Amazon, Google veri merkezi.

Saydamlık (transparency): Kullanıcı, sistemin kaç parçadan oluştuğunu veya bu parçaların nerede olduğunu bilmez, sadece sisteme bir girdi verir ve çıktıyı alır.

Heterojen Otomata: Sistemdeki makineler aynı olmak zorunda değildir; farklı işletim sistemleri veya donanımlar bir arada uyum içinde çalışır.

Location Transparency (Konum Saydamlığı): Bir dosyanın veya işlemin hangi sunucuda olduğu kullanıcıyı ilgilendirmez. Kullanıcı "dosyayı aç" der, sistem o dosyanın hangi kütadaki sunucuda olduğunu kendi halleder.

Storage Replication & Load Balancing: Verinin kaç kopyasının olduğu veya sistemin çok yoğunlaştığında işi diğer makinelere nasıl dağıttığı kullanıcıya hissettirilmez.

İstemci-Sunucu (Client-Server) Yapısı: Genellikle kullanıcı (istemci) bir talep gönderir ve arka plandaki karmaşık yapı (sunucular grubu) bu talebi işleyip basit bir yanıt döndürür.

- Bilgisayarlar birbirleriyle veri alışverişi yapabiliyorsa "bağlı" sayılırlar. Bu sadece kabloyla değil; fiber optik, mikrodalga veya uydu ile de olabilir.
- Otonomluk, bilgisayarlar birbirinden bağımsız olmalıdır. Eğer bir bilgisayar diğerini zorla çalıştırıp durdurabiliyorsa bu bir ağ değil, tek merkezli bir sistemdir.
- Bir dağıtık sistem, bir ağın üzerine giydirilmiş akıllı bir yazılım katmanıdır. Farkı oluşturan şey donanım değil, **işletim sistemidir**.
- Dağıtık sistemin, tasarımı, uygulaması, bakımı, algoritması önemlidir bunlara dikkat ederiz.

Aşağıda listelenen avantajlar, neden devasa ve pahalı tek bir bilgisayar (mainframe) yerine bu yapıları kullandığımızı açıklıyor:

- Kaynak Paylaşımı (Resource Sharing):** Yazıcı, veri seti veya donanım gibi kaynakların ortak kullanılması.
- Yüksek Güvenilirlik (High Reliability):** Bir makine bozulursa diğerleri işi devralır; sistem tamamen çökmez.
- Maliyet Tasarrufu (Saving Money):** Çok pahalı tek bir dev bilgisayar yerine, daha ucuz birçok küçük bilgisayarın (Client-Server modeli) kullanılması.
- Ölçeklenebilirlik (Scalability):** İhtiyaç arttıkça sisteme yeni makineler ekleyerek performansı kademeli olarak artırılabilir.
- İletişim Ortamı:** İnsanlar arası iletişimi (e-posta, mesajlaşma vb.) sağlama.

BİLGİSAYAR AĞLARI VE DAĞITIK SİSTEMLER **ARASINDAKİ SOMUT FARK ÖRNEĞİ**

1. Bilgisayar Ağı (Network) Örneği: "Dosya Göndermek"

Bir arkadaşına e-posta veya Bluetooth ile dosya gönderdiğini düşün.

- **Senin Rolün:** Bilgisayarını açarsın, arkadaşının bilgisayarını veya adresini seçersin, "Gönder" tuşuna basarsın.
- **Farkındalık:** Dosyanın senin bilgisayarından çıktığını ve onun bilgisayarına gittiğini bilirsin. Eğer arkadaşının bilgisayarı kapalıysa, dosyanın gitmeyeceğini de bilirsin.
- **Yönetim:** Dosyanın nerede duracağına (C sürücüsü mü, Masaüstü mü) sen veya arkadaşın karar verir.
- **Özet:** Burada "**Açıkça**" (**Explicitly**) hareket edersin. İki ayrı makine olduğunun ve aradaki bağlantının farkındasın.

2. Dağıtık Sistem Örneği: "Google Drive veya iCloud"

Şimdi bir dosyayı Google Drive klasörüne sürüklediğini düşün.

- **Senin Rolün:** Dosyayı sadece klasöre bırakırsın. Senin için o dosya artık "Drive"dadır.
- **Sistemin Rolü (Saydamlık):** Arka planda Google'ın sistemi bu dosyayı belki Hollanda'daki bir sunucuya, belki de bir yedeğini Amerika'daki başka bir sunucuya taşır. Eğer Hollanda'daki sunucu bozulursa, sistem dosyayı otomatik olarak başka bir yere taşır.
- **Senin Farkındalığın:** Sen bu taşınma işlemlerinin hiçbirini görmezsin. Senin için sadece "dosyam orada" bilgisi vardır. Dosyanın fiziksel olarak hangi bilgisayarın diskinde döndüğünü bilmezsin ve bilmene de gerek yoktur.
- **Özet:** Burada her şey "**Kendiliğinden**" (**Automatically**) olur. Sistem sana tek bir devasa sabit diskmiş gibi görünür.

Bu görsel, insanların birbirleriyle olan sosyal bağlarını (arkadaşlıklarını) gösteriyor.

- **Neden Dağıtık Sistem Değildir?** Çünkü buradaki düğümler (nodes) bilgisayarlar değil, **insanlardır**. İnsanlar otonomdur ve bir ağ oluştururlar (social network), ancak bir "işlem" yapmak için bir araya gelip tek bir sanal makine gibi davranmazlar.
- **Tanım:** Bu bir **Bilgisayar Ağı (Network)** veya **Sosyal Ağ** örneğidir. Kullanıcılar (insanlar) birbirlerinin varlığından haberdardır ve etkileşimi bizzat (explicitly) yönetirler.

(B) Peer-to-Peer (P2P) Dosya Paylaşım Sistemi (Gnutella)

İşte bu, teknik anlamda gerçek bir **Dağıtık Sistem** örneğidir.

Neden Dağıtık Sistemdir?

- 1. **Şeffaflık:** Sen Gnutella veya benzeri bir P2P ağında (Torrent gibi) bir dosya aradığında, o dosyanın hangi kullanıcının bilgisayarında olduğunu, hangi ülkeden geldiğini veya parçalarının nasıl birleştiğini görmezsin.
- 2. **Tek Bir Sistem Algısı:** Sen sadece "dosyayı indir" dersin. Arka plandaki yüzlerce **otonom** bilgisayar, sanki tek bir devasa kütüphaneymiş gibi çalışıp dosyayı sana sunar.
- 3. **Hata Toleransı:** Ağdaki 10 kişi bilgisayarını kapatsa bile (hata/kopma olsa bile), sistem çalışmaya devam eder ve sen bunu hissetmezsin.

Uygulama	Uygulama Katmanı Protokolü (Kural)	Taşıma Protokolü (Altyapı)
E-posta	SMTP (Posta gönderim kuralı)	TCP (Güvenilir iletim)
Web (Tarayıcı)	HTTP (Web sayfası kuralı)	TCP (Güvenilir iletim)

Pdfteki facebook da gezinen insanlar mı dağıtık yoksa peer to peer dosya paylaşım sistemi gnutella mı?

(A) Facebook Sosyal Ağı (İnsanlar Arasındaki İlişkiler)

Uygulama	Uygulama Katmanı Protokolü (Kural)	Taşıma Protokolü (Altyapı)
Dosya Aktarımı	FTP	TCP
Video Akışı	Özel (Proprietary) (Örn: RealNetworks)	TCP veya UDP
İnternet Telefonu	Özel (Proprietary) (Örn: Skype)	Genellikle UDP (Hız odaklı)

HTTP

1. World Wide Web'in Kalbi: HTTP Standardi

Bu görselde HTTP'nin temel tanımı ve **İstemci/Sunucu (Client/Server)** modeli tanıtılıyor.

- **HTTP Nedir?:** "Hypertext Transfer Protocol"ün kısaltmasıdır. Web'in "uygulama katmanı" (application layer) protokolüdür.
- **İstemci (Client):** Web nesnelerini (sayfalar, görseller vb.) talep eden, alan ve ekranında gösteren tarayıcıdır (Örn: Chrome, Safari).
- **Sunucu (Server):** Web sitesini barındıran, gelen isteklere karşılık ilgili nesneleri gönderen yapıdır (Örn: Apache Web Server).
- **Standartlar:** HTTP 1.0 (RFC 1945) ve HTTP 1.1 (RFC 2068) sürümleri mevcuttur. HTTP 1.1, aynı bağlantı üzerinden birden fazla nesneyi (resimler, scriptler) indirmeyi sağlayarak daha verimlidir.
- **Şemadaki Akış:** Farklı cihazlar (PC'de Internet Explorer veya Mac'te Safari) sunucuya bir **"http request" (istek)** gönderir, sunucu ise buna **"http response" (yanıt)** ile karşılık verir.

2. HTTP Protokolü: Daha Fazlası (Stateless Yapısı)

Bu bölümde HTTP'nin çalışma prensibindeki iki kritik teknik özellik anlatılıyor: **TCP Kullanımı** ve **Stateless (Durumsuzluk)**.

• HTTP ve TCP İlişkisi:

- İstemci, sunucunun **80. portuna** bir **TCP bağlantısı** başlatır (bir "socket" oluşturur).
- Sunucu bu bağlantıyı kabul eder.
- Tarayıcı ve sunucu arasında HTTP mesajları (istek ve yanıt) bu güvenli hat üzerinden teati edilir.
- İşlem bitince TCP bağlantısı kapatılır.

• HTTP "Stateless"dır (Durumsuzdur):

- Sunucu, istemcinin geçmişteki isteklerine dair hiçbir bilgi tutmaz. Her istek, sanki ilk kez yapılmış gibi sıfırdan değerlendirilir.

• Neden Stateless?:

- **Karmaşıklığı önler:** "State" (durum) tutan protokoller çok karmaşıktır; geçmiş geçmiş verilerin sürekli güncellenmesi gerekir.
- **Hata toleransı:** Eğer sunucu veya istemci çökerse, ellerindeki "durum" bilgileri tutarsız hale gelir ve bunları düzeltmek çok zordur. RESTful protokollerin çoğu bu yüzden stateless'tir.

3. HTTP Örneği (Adım Adım Süreç)

Bir kullanıcının bir URL girdiğinde arka planda neler olduğu zaman çizelgesiyle (time) anlatılıyor.

- **Senaryo:** Kullanıcı tarayıcıya www.cs.uiuc.edu/ adresini yazar (bu sayfa metin ve 10 adet resim referansı içermektedir).
- **Adım 1a & 1b:** HTTP istemcisi, sunucunun 80. portuna bir TCP bağlantısı açma isteği gönderir. Sunucu bu isteği beklemektedir ve bağlantıyı kabul ederek istemciye bildirir.
- **Adım 2:** İstemci, istenen URL'yi içeren bir **HTTP request message (istek mesajı)** hazırlayıp TCP soketi üzerinden gönderir.
- **Adım 3:** Sunucu isteği alır, istenen nesneyi (index.html) bulur, bir **response message (yanıt mesajı)** oluşturur ve soket üzerinden geri gönderir.

4. HTTP Örneği (Devamı ve Bağlantı Türleri)

İşlemin nasıl sonuçlandığı ve verimlilik seçenekleri anlatılıyor.

- **Adım 4:** Sunucu (gerekli durumlarda) TCP bağlantısını kapatır.
- **Adım 5:** İstemci index.html dosyasını alır ve ekranda gösterir. HTML dosyasını analiz ederken (parsing) içinde 10 tane daha resim (jpeg) dosyasına referans olduğunu bulur.
- **Adım 6:** İçerideki o 10 resim için **1'den 5'e kadar olan adımlar her bir resim için tekrarlanır.**
- **Nesneleri Getirmek İçin 2 Seçenek:**
 1. **Non-persistent connection (Kalıcı olmayan):** Her bir nesne için ayrı bir TCP bağlantısı açılır. Bazı tarayıcılar hızı artırmak için aynı anda birden fazla TCP bağlantısı açabilir.
 2. **Persistent connection (Kalıcı):** Tek bir TCP bağlantısı açılır ve tüm nesneler (HTML + 10 resim) aynı hat üzerinden sırayla gönderilir. Bu yöntem çok daha performanslıdır.

Özetle: HTTP, basit bir "istek-yanıt" mantığıyla çalışır, güvenilir iletim için TCP'yi kullanır ve sunucunun iş yükünü azaltmak için "geçmiş hatırlamayacak" (stateless) şekilde tasarlanmıştır.

"Önemli" Dağıtık Sistem Sorunları/Zorlukları

Dağıtık sistemleri tasarlamayı zor (ve eğlenceli) kılan gerçek dünya problemleridir:

- **Evrensel Saat Yok (No global clock):** Sistemdeki tüm makinelerin ortak bir "doğru zaman" algısı yoktur. İşlemler **asenكرون** gerçekleşir; yani kimin önce neyi yaptığını belirlemek zordur.
- **Öngörülemez Bileşen Hataları:** Bir yanıt alamadığınızda bunun sebebi; bir ağ kablosunun kopması, sunucunun çökmesi veya yol üzerindeki bir router'ın bozulması olabilir. Sistem **güvenilmez (unreliable)** kabul edilir.
- **Değişken Bant Genişliği (Bandwidth):** Veri iletim hızı çok değişkendir. 16 Kbps (çok yavaş modemler veya Google Balonu gibi projeler) ile Gbps (Internet2) veya Tbps (şirket içi devasa veri merkezleri) arasında devasa farklar olabilir.

- **Yüksek ve Değişken Gecikme (Latency):** Bir mesajın ulaşması birkaç milisaniye sürebileceği gibi saniyeler de sürebilir. Bu belirsizlik tasarımı zorlaştırır.
- **Çok Sayıda Host:** Sistem sadece 2 bilgisayardan oluşabileceği gibi, milyonlarca bilgisayar (host) kapsayacak kadar devasa olabilir.

Özetle: Dağıtık sistemler; her an bozulabilecek parçalardan oluşan, ortak bir saati olmayan ve hızı sürekli değişen ağlar üzerinde "tek bir makineymiş gibi" sorunsuz çalışma sanatıdır.

DESIGN GOAL

1. Heterogeneity (Çeşitlilik/Heterojenlik)

Sistemin farklı türdeki cihaz ve teknolojilerle uyumlu çalışabilme yeteneğidir.

- **Tanım:** Sistem; farklı donanımlar (PC, telefon, tablet), farklı işletim sistemleri ve farklı ağ protokollerini bir arada idare edebilir mi?
- **Teknikler:** Farklı dillerin haberleşmesini sağlayan **Middleware** (Ara katman) yapıları kullanılır.
- **Örnek: gRPC** gibi modern frameworkler sayesinde Python ile yazılmış bir servis, Java ile yazılmış bir servis ile sorunsuz konuşabilir.

2. Robustness (Dayanıklılık/Sağlamlık)

Sistemin beklenmedik aksaklıklara karşı direnç göstermesidir.

- **Tanım:** Bir bilgisayar çöktüğünde veya ağdaki bazı mesajlar kaybolduğunda sistem ayakta kalmaya devam edebiliyor mu?
- **Teknikler: Fault tolerance** (hata toleransı), bir lider çöktüğünde yenisini seçen **Leader election** algoritmaları ve sistemin durumunu yedekleyen **Checkpointing** kullanılır.
- **Örnek: Paxos** ve **Raft** gibi konsensüs algoritmaları bu dayanıklılığı sağlar.

3. Scalability (Ölçeklenebilirlik)

Sistemin artan yük altında performansını koruyabilmesidir.

- **Tanım:** Sistem 100 milyon kullanıcıyı, hatta 6 milyar kullanıcıyı hizmet kalitesi düşmeden idare edebilir mi?
- **Teknikler: Horizontal scaling** (sisteme daha fazla makine eklemek), **Load balancing** (yükü

makineler arasında paylaşmak) ve veriyi parçalara bölmek (**Sharding**).

- **Örnek: Kubernetes** konteyner yönetimi için, **MongoDB** ise veri sharding'i için kullanılır.

4. Availability (Erişilebilirlik)

Hizmetin her an kullanıma hazır olması durumudur.

- **Tanım:** Veri ve servisler istemciler için her zaman orada mı?
- **Teknikler:** Verinin kopyalarını çıkarma (**Replication**), ana sistem çöktüğünde yedeğe geçme (**Failover**) ve meşhur **CAP teoremi** prensipleri.
- **Örnek:** Amazon'un **AWS Multi-AZ** (Çoklu Erişilebilirlik Alanı) yapısı, bir veri merkezi yansa bile servisin kesilmemesini sağlar.

5. Transparency (Saydamlık)

Kullanıcının sistemin karmaşıklığından haberdar olmamasıdır.

- **Tanım:** Sistem, kendi iç işleyişini (dosyaların nerede olduğu, hangi işlemcinin çalıştığı vb.) kullanıcıdan gizleyebiliyor mu?
- **Örnek: NFS** (Network File System) veya **Google File System**. Dosya aslında binlerce farklı diskte parçalanmış haldedir ama sen onu tek bir klasör gibi görürsün.

6. Concurrency (Eşzamanlılık)

Aynı anda gelen çoklu isteklerin yönetilmesidir.

- **Tanım:** Sunucu, binlerce farklı istemciye aynı anda, verileri birbirine karıştırmadan hizmet verebilir mi?
- **Teknikler:** Veriyi kilitleme (**Locks**), iyimser eşzamanlılık kontrolü ve verinin eninde sonunda her yerde aynı olmasını sağlayan **Eventual consistency**.
- **Örnek: DynamoDB** ve **Cassandra** gibi NoSQL veritabanları bu konuda ustadır.

7. Efficiency (Verimlilik)

Kaynakların en akılcı şekilde kullanılmasıdır.

- **Tanım:** Servis yeterince hızlı mı? Kaynakların (CPU, RAM) %100'ünü verimli kullanabiliyor mu?
- **Teknikler:** Veriyi önbelleğe alma (**Caching**), veriyi mantıksal bölümlere ayırma (**Partitioning**) ve veri sıkıştırma (**Compression**).

- **Örnek: Redis** veya **Memcached** gibi bellek içi (in-memory) sistemler hızı artırmak için kullanılır.

8. Security (Güvenlik)

Sistemin kötü niyetli saldırılara karşı korunmasıdır.

- **Tanım:** Sistem hacker saldırılarına karşı koyabiliyor mu?
- **Teknikler:** Şifreleme (**Encryption**), kimlik doğrulama (**Authentication**) ve saldırı tespit sistemleri (**IDS**).
- **Örnek: TLS** (şifreli iletişim), **OAuth** (güvenli giriş) sistemleri.

9. Openness (Açıklık)

Sistemin genişletilebilir ve esnek olmasıdır.

- **Tanım:** Sisteme yeni özellikler veya donanımlar kolayca eklenebilir mi?
- **Teknikler:** Açık standartlar ve sistemlerin birbiriyle çalışabilmesi (**Interoperability**).
- **Örnek: REST API**'ler ve bulut yönetim platformu olan **OpenStack**.

10. Failure Handling (Hata Yönetimi)

Hataların nasıl tespit edildiği ve düzeltildiğidir.

- **Tanım:** Sistem hata yapmaya ne kadar meyilli ve bu hatalar nasıl yönetiliyor?
- **Teknikler:** Hata tespiti (**Failure detection**), sistemin kendi kendini onarması (**Self-healing**) ve Bizans hata toleransı (**BFT**).
- **Örnek: Kubernetes** çöken bir konteyneri otomatik olarak yeniden başlatarak "kendi kendini onarma" (self-healing) yapar.

Özetle:

Bu hedefler bazen birbirleriyle çelişir (örneğin güvenliği artırmak verimliliği biraz düşürebilir). İyi bir mühendis olarak senin görevin, projenin ihtiyacına göre (WhatsApp mı yapıyorsun, bankacılık sistemi mi?) bu dengeleri doğru kurmaktır.

Temel Konsensüs Mekanizmaları

- **Proof of Work (PoW - İş Kanıtı):**
 - **Mantık:** Karmaşık matematiksel bulmacaları çözen ilk kişi bloğu doğrular.

- **Artısı:** Çok güvenli ve tamamen merkezsizdir.
- **Eksisi:** Aşırı enerji tüketir ve yavaşır (Örn: Bitcoin).
- **Proof of Stake (PoS - Hisse Kanıtı):**
 - **Mantık:** Elinde en çok jeton (token) tutan ve bunları kilitleyenlerin doğrulama şansı artar.
 - **Artısı:** Enerji dostu ve daha hızlıdır.
 - **Eksisi:** Zenginlerin daha çok söz sahibi olması (servet konsantrasyonu) riski vardır (Örn: Ethereum 2.0).
- **DPOs (Delegated Proof of Stake):**
 - **Mantık:** Jeton sahipleri, blokları doğrulaması için temsilcilere oy verir.
 - **Artısı:** Çok hızlı ve ölçeklenebilirdir.
 - **Eksisi:** Az sayıda temsilci olması merkezileşme riski doğurur (Örn: TRON).
- **PoA (Proof of Authority):**
 - **Mantık:** Kimliği önceden onaylanmış, güvenilir otoriteler doğrulamayı yapar.
 - **Artısı:** Çok hızlı ve verimlidir.
 - **Eksisi:** Doğası gereği merkezidir (Örn: Özel kurumsal ağlar).
- **PoH (Proof of History):**
 - **Mantık:** İşlemlerin sırasını kanıtlamak için kriptografik bir saat kullanılır.
 - **Artısı:** Çok yüksek işlem hacmi ve düşük gecikme sağlar.
 - **Eksisi:** Yeni bir teknoloji olduğu için daha az test edilmiştir (Örn: Solana).
- **PoSpace / PoCapacity:**
 - **Mantık:** Doğrulama için bilgisayarın boş disk alanı tahsis edilir.
 - **Artısı:** Enerji verimlidir, mevcut donanımı kullanır (Örn: Chia).
- **PoB (Proof of Burn - Yakma Kanıtı):**
 - **Mantık:** Doğrulama hakkı kazanmak için jetonlar kalıcı olarak "yakılır" (yok edilir).
 - **Artısı:** Gereksiz biriktirmeyi engeller.
- **PoET (Proof of Elapsed Time):**

- **Mantık:** Doğrulayıcılar arasında rastgele bir bekleme süresi atanır; süresi ilk biten bloğu yazar.
- **Artısı:** Adil ve enerji dostudur (Örn: Hyperledger Sawtooth).
- **BFT (Byzantine Fault Tolerance):**
 - **Mantık:** Ağdaki bazı düğümler kötü niyetli olsa veya yanlış bilgi verse bile sistemin doğru karara varmasını sağlar.
 - **Artısı:** Hızlı ve güvenlidir.
 - **Eksisi:** Düğüm sayısı arttıkça iletişim trafiği (overhead) çok artar (Örn: Ripple).
- **DAG (Directed Acyclic Graph):**
 - **Mantık:** Bloklar yerine, her yeni işlem kendinden önceki iki işlemi doğrular; zincir değil, ağ yapısıdır.
 - **Artısı:** Madencisiz, ücretsiz ve çok ölçeklenebilir (Örn: IOTA).
- **PoA (Proof of Activity):**
 - **Mantık:** PoW ve PoS'un bir karışımıdır; önce bulmaca çözülür, sonra rastgele seçilen hisse sahipleri imzalar.
 - **Artısı:** Güvenlik ve enerji dengesi kurar.
- **Pol (Proof of Importance):**
 - **Mantık:** Sadece ne kadar paran olduğuna değil, ağdaki aktifliğine ve kimlerle işlem yaptığına göre puan verilir.
 - **Artısı:** Katılımı teşvik eder (Örn: NEM).
- **Pol (Proof of Identity):**
 - **Mantık:** Kullanıcının gerçek kimliğine dayalı doğrulama yapılır.
 - **Artısı:** Güvenlidir, sahte hesap saldırılarını (Sybil) önler.
- **PoR (Proof of Reputation):**
 - **Mantık:** Katılımcıların geçmişteki dürüstlüklerine (itibar puanına) bakılır.
 - **Artısı:** Güveni teşvik eder.
- **PoWeight:**

- **Mantık:** Doğrulama yetkisi çeşitli ağırlıklı puanlara göre belirlenir.
- **Artısı:** Esnektir ve özelleştirilebilir (Örn: Algorand).

Özetle:

Bu mekanizmaların her biri, dağıtık bir sistemin **güvenlik, hız ve merkezsizlik** arasındaki dengesini farklı şekillerde kurmaya çalışır. Hangi mekanizmanın seçileceği, sistemin ne kadar hızlı olması gerektiğine veya ne kadar güvenliğe ihtiyaç duyduğuna bağlıdır.

BULUT BİLİŞİM

Bulut Nedir? (Tanım ve Mimari)

Bu slayt, "Bulut" kavramının ne olduğunu sorgulayarak başlıyor.

- **Bulut nedir?:** Bir küme (cluster) mi? Bir süper bilgisayar mı? Bir veri deposu mu? Yoksa Süpermen mi?
- **Cevap:** Yukarıdakilerin hiçbiri veya hepsi! Teknik olarak **Bulut = Elinizin altında bulunan devasa miktarda depolama ve işlem gücü (compute cycles) demektir.**
- **Veri Merkezi (Datacenter) Mimarisi:** Tek bir bölgedeki (single-site) bulut şu parçalardan oluşur:
 - **İşlem Dğümleri (Compute nodes):** Sunucu rafları (racks) içinde gruplanırlar.
 - **Anahtarlar (Switches):** Bu rafları birbirine bağlar.
 - **Ağ Topolojisi:** Genellikle hiyerarşik bir yapıdadır.
 - **Depolama (Backend) Dğümleri:** Ağa bağlı veri saklama birimleri.
 - **Ön Yüz (Front-end):** İstemci taleplerini almak ve işleri sisteme iletmek için kullanılır.
 - **Üç Katmanlı Mimari (Three-tier architecture):** Yukarıdaki bileşenlerin (1-3) oluşturduğu yapıdır.
 - **Yazılım Servisleri:** Sistemin üzerinde çalışan uygulamalar.
- **Coğrafi Olarak Dağıtık Bulut:** Birden fazla veri merkezinin birleşimidir; her bir bölge farklı bir yapıya ve servislere sahip olabilir.

Günümüz Bulutlarında Yeni Olan Dört Özellik

Modern bulut sistemlerini tanımlayan dört temel sütun şunlardır:

1. **Massive Scale (Devasa Ölçek):** Sistemler milyonlarca bilgisayarı kapsayacak kadar büyüktür.
2. **On-demand Access (İsteğe Bağlı Erişim):** "Kullandığın kadar öde" (Pay-as-you-go) modeli hakimdir. Önceden büyük bir taahhüt vermeniz gerekmez; herkes bu kaynaklara anında erişebilir.
3. **Data-intensive Nature (Veri Yoğunluklu Yapı):** Eskiden MB'lar ile ölçülen veriler artık TB (Terabayt), PB (Petabayt) ve EB (Egzabayt) seviyelerine ulaştı.
 - *İlginç Not:* Wikipedia'nın tüm metin içeriği sıkıştırıldığında sadece yaklaşık 10 GB yer kaplar; ancak bulutlar günlük loglar, adli veriler ve web verileriyle bunun milyonlarca katını işler.
4. **Yeni Bulut Programlama Paradigmaları:** MapReduce/Hadoop, NoSQL (Cassandra, MongoDB) gibi yeni yöntemler doğmuştur. Bu araçlar yüksek erişilebilirlik ve programlama kolaylığı sunar, ayrıca çoğu açık kaynaklıdır.

II. On-demand Access: *aaS Sınıflandırması

Bulut bilişimde "isteğe bağlı erişim", bir araba satın almak yerine ihtiyaç duyduğunda taksi çağırmaya benzetilir. Hizmetler, kullanıcının ne kadar kontrol sahibi olduğuna göre katmanlara ayrılır:

1. HaaS (Hardware as a Service)

- **Tanım:** Kullanıcıya doğrudan yalın donanım (barebones) makineler sunulur.
- **Örnek:** Kendi bilgisayar kümenizi (cluster) kurmak gibi.
- **Not:** Güvenlik riskleri nedeniyle her zaman iyi bir fikir olmayabilir.

2. IaaS (Infrastructure as a Service - Altyapı)

- **Tanım:** Esnek hesaplama ve depolama altyapısına erişim sağlar. Bu model genellikle sanallaştırma teknolojileri (**Docker, Kubernetes, VM'ler**) üzerine kuruludur ve HaaS'ı kapsar.
- **Örnek:** Amazon AWS (EC2, S3), Microsoft Azure, Google Cloud.

3. PaaS (Platform as a Service - Platform)

- **Tanım:** Altyapının yanı sıra, yazılım geliştirme için hazır bir platform (işletim sistemi, programlama dili çalışma ortamı) sunulur. Geliştirici sadece kodu yazar; sunucu yönetimiyle ilgilenmez.
- **Örnek:** Google AppEngine (Python, Java, Go desteğiyle).

4. SaaS (Software as a Service - Yazılım)

- **Tanım:** Kullanıcının doğrudan internet üzerinden bir yazılımı hizmet olarak kullanmasıdır. Hizmet Odaklı Mimari (SOA) kavramını kapsar.
- **Örnek:** Google Docs, MS Office 365.

III. Data-intensive Computing (Veri Yoğunluklu Hesaplama)

Bilgisayar dünyasında odağın "hesaplama gücünden" "veriyi" kaydığı iki farklı yaklaşım:

- **Computation-Intensive (Hesaplama Yoğunluklu):** Çok karmaşık matematiksel işlemler yapılır. Genellikle süper bilgisayarlar (Örn: NCSA Blue Waters) veya Grid yapılarında çalışır.
- **Data-Intensive (Veri Yoğunluklu):** Verinin boyutu o kadar büyüktür ki, veriyi işlemciye getirmek yerine **hesaplamayı verinin yanına götürmek** gerekir.
 - **Kritik Fark:** Veri yoğunluklu sistemlerde en önemli metrik artık CPU kullanımı değil, **I/O (Giriş/Çıkış)** hızıdır (Disk ve ağ hızı).

IV. New Cloud Programming Paradigms (Yeni Bulut Programlama Modelleri)

Devasa boyutlardaki veriyi paralel olarak işleyebilmek için geliştirilen yeni yöntemler:

- **MapReduce & Hadoop:** Veriyi parçalara bölüp (Map) farklı makinelerde işleyen ve sonra sonuçları birleştiren (Reduce) sistemlerdir.
 - **Google:** 2006 yılında ayda 200 bin iş (job) ile 50 PB veri işliyordu.
 - **Yahoo! (Hadoop + Pig):** 2008'de 300 TB veriyi 10 bin çekirdek ile işliyordu.

- **Facebook (Hadoop + Hive):** 2008'de günde 2 TB veri ekleyerek toplam 300 TB veriyi yönetiyordu.
- **NoSQL Devrimi:**
 - Endüstri standardı olan MySQL yerine, dağıtık yapıda çalışan **Cassandra** gibi sistemlerin bazı senaryolarda **2400 kat daha hızlı** olduğu vurgulanıyor.

Özetle:

Bu slaytlar, bulutun sadece "başkasının bilgisayarı" olmadığını; yazılım geliştirme şeklimizi (PaaS/SaaS), önceliklerimizi (CPU'dan I/O'ya) ve veriyi işleme hızımızı (MapReduce/NoSQL) kökten değiştiren bir **ekosistem** olduğunu kanıtlıyor.

1. Bulut Kategorileri: Public vs. Private

İki ana bulut türü arasındaki temel farklar hatırlatılıyor:

- **Public Cloud (Genel Bulut):** Ödeme yapan her müşteriye açıktır (Örn: AWS, Google Cloud).
- **Private Cloud (Özel Bulut):** Sadece şirket çalışanlarına açıktır.
- **Stratejik Soru:** Yeni bir şirket kuruyorsanız, genel bulutu mu kullanmalısınız yoksa kendi özel bulut donanımınızı mı satın almalısınız? Cevap aşağıdaki maliyet analizinde saklı.

2. Maliyet Analizi: Dış Kaynak (Outsource) mı, Kendi Sistemin (Own) mi?

Orta ölçekli bir organizasyonun \$M\$ ay boyunca bir servis çalıştırmak istediğini varsayalım.

- **İhtiyaç:** 128 sunucu (1024 çekirdek) ve 524 TB depolama.

Dış Kaynak Kullanımı (Örn: AWS üzerinden kiralama - 2009 fiyatları):

- **Depolama Maliyeti:** $\$0.12 \times 524 \times 1000 \approx \mathbf{\$62.000}$ (aylık).
- **İşlem (CPU) Maliyeti:** $\$0.10 \times 1024 \times 24 \times 30 \approx \mathbf{\$74.000}$ (aylık).
- **Toplam Aylık Maliyet:** Yaklaşık **\$136.000**.

Kendi Sistemine Sahip Olma (Satın Alma):

- **Donanım Alımı:** Toplamda yaklaşık $\$1.555.000$ (3 yıllık donanım ömrü baz alınmış).

- **İşletme Giderleri:** Ayda \$7.500\\$\$ (her 100 düğüm için 1 sistem yöneticisi maaşı dahil).
- **Aylık Maliyet Formülü:** $(\$1555K / M) + 7.5K\$$

3. Başa Baş Noktası Analizi (Breakeven Analysis)

Hangi noktadan sonra satın almak kiralamaktan daha mantıklı hale gelir?

- **Depolama için:** Eğer $\$M > 5.55\$$ ay ise satın almak daha ucuzdur.
- **Toplam Sistem için:** Eğer $\$M > 12\$$ ay ise kendi donanımını satın almak, buluttan kiralamaktan daha karlı hale gelir.

Sonuç: 1. **Startuplar bulutu çok kullanır:** Çünkü başlangıçta milyonlarca dolarlık donanım yatırımı (Capex) yapacak bütçeleri yoktur.

2. **Bulut sağlayıcılar en çok depolamadan para kazanır:** Depolama maliyetleri kullanıcı için çok hızlı katlanır.

4. Akademik Bulutlar: Emulab

Özel bir bulut/küme (cluster) örneği olarak **Emulab** tanıtılıyor:

- **Nedir?** Akademisyenlerin ve endüstri araştırmacılarının deneyleri için açık olan bir topluluk kaynağıdır.
- **Konum:** Utah Üniversitesi tarafından kurulmuştur ve yönetilmektedir. Yaklaşık 500 sunucu barındırır.
- **Kullanıcı Hakları:**
 - Deneyler için bir grup makineyi rezerve edebilir.
 - Makinelere **root (sudo)** seviyesinde tam erişim sağlar.
 - Kümenizin **ağ topolojisini** (makinelere birbirine nasıl bağlanacağını) bizzat belirleyebilirsiniz.
 - İstedığınız her türlü ağ yapısını simüle (emulate) edebilirsiniz.

Özetle:

Eğer projeniz kısa süreliyse veya hızlıca prototip üretmek istiyorsanız **Public Cloud** (SaaS/IaaS) en iyisidir. Ancak uzun vadeli (1 yıldan fazla) ve çok yüksek kapasiteli bir iş yapıyorsanız, maliyetler

nedeniyle kendi **Private Cloud** yapınızı kurmak daha mantıklı hale gelir. Emulab gibi yapılar ise araştırmacılara bulutun gücünü, satın alma maliyeti olmadan ve tam kontrolle sunan "ara" çözümlerdir.

MAPREDUCE AND HADOOP

1. Map (Haritalama) Aşaması

Map aşaması, verinin ham halden anlamlı **key/value** (anahtar/değer) çiftlerine dönüştürüldüğü ilk adımdır.

- **Bireysel Kayıt İşleme:** Map fonksiyonu, her bir kaydı (örneğin bir metin belgesindeki satırları) bağımsız olarak işler.
- **Key/Value Üretimi:** "Welcome Everyone" satırı girdiğinde, Map bunu Welcome: 1 ve Everyone: 1 şeklinde ara değerlere böler.
- **Parallelleştirme:** Devasa veriler "Split" edilerek binlerce farklı **Map Task**'e bölünür. Bu görevler farklı sunucularda aynı anda (paralel) çalışır.
- **Hadoop Kodu (Map):** Kodda görüldüğü üzere map fonksiyonu her satırı string'e çevirir, kelimelere böler (Tokenizer) ve her kelime için `output.collect(word, one)` komutuyla anahtarı kelime, değeri ise "1" olan bir çift oluşturur.

2. Shuffle ve Reduce (Birleştirme) Aşaması

Map'ten çıkan ara değerlerin toplanıp özetlendiği kısımdır.

- **Shuffle (Karıştırma):** Map çıktıları, aynı anahtara (key) sahip olanlar aynı **Reduce Task**'ine gidecek şekilde gruplanır.
- **Reduce İşlemi:** Belirli bir anahtara ait tüm "1" değerleri toplanır. Örneğin, farklı Map'lerden gelen Everyone: 1 ve Everyone: 1 değerleri, Reduce aşamasında Everyone: 2 sonucuna dönüştürülür.
- **Partitioning (Bölümleme):** Hangi anahtarın hangi Reduce görevine gideceği genellikle bir **Hash** fonksiyonu (`hash(key) % task_sayısı`) ile belirlenir. Bu, iş yükünün dengeli dağılmasını sağlar.
- **Hadoop Kodu (Reduce):** reduce fonksiyonu bir anahtar ve o anahtara ait değerlerin listesini (Iterator) alır. Bir döngü içinde tüm "1"leri toplar (`sum += values.next().get()`) ve final sonucu yazar.

MapReduce'un İç Yapısı ve Mekaniği

Sistemin mimari olarak nasıl ayakta kaldığını açıklar:

- **Barrier (Bariyer):** Çok kritik bir kuraldır; **tüm Map görevleri bitmeden hiçbir Reduce görevi başlayamaz.** Arada bir engel (barrier) vardır.
- **Veri Transferi:** Map çıktıları yerel diske (Local FS) yazılır. Reduce görevleri bu veriyi ağ üzerinden okur (**Local write, remote read**). Final çıktı ise dağıtık dosya sistemine (HDFS veya GFS) kalıcı olarak kaydedilir.
- **Resource Manager:** Hangi Map ve Reduce görevlerinin hangi sunucuda çalışacağına karar veren yöneticidir.
- **Hata Toleransı (Eksik Bilgi Tamamlama):** Slaytlarda açıkça belirtilmese de, MapReduce'un en büyük gücü bir sunucu çöktüğünde o görevi otomatik olarak başka bir sunucuda tekrar başlatabilmesidir (Self-healing).

Özet Mimari Akışı:

1. Veri blokları **HDFS**'den okunur.
2. **Mapper**'lar yerel makinelerde veriyi işler.
3. **Shuffle** ile veriler ağ üzerinden Reducer'lara taşınır.
4. **Reducer**'lar sonuçları birleştirir ve tekrar **HDFS**'e yazar.

MapReduce Özeti ve Akış Şeması

- **Özet:** MapReduce, paralel işleme ve sonuçları birleştirme (aggregation) mantığıyla çalışır. Hatalarla başa çıkmak sistemin doğasında vardır.
- **Akış Şeması (Word Count Örneği):**
 1. **Input:** Ham veri girer (Bear, River, Car vb.).
 2. **Splitting:** Veri parçalara bölünür.
 3. **Mapping:** Her parça <Kelime, 1> ikililerine dönüştürülür.
 4. **Shuffling:** Aynı kelimeler aynı kutulara gidecek şekilde gruplanır.

5. **Reducing:** Kelime sayıları toplanır.
6. **Final Result:** Sonuç listesi oluşturulur.